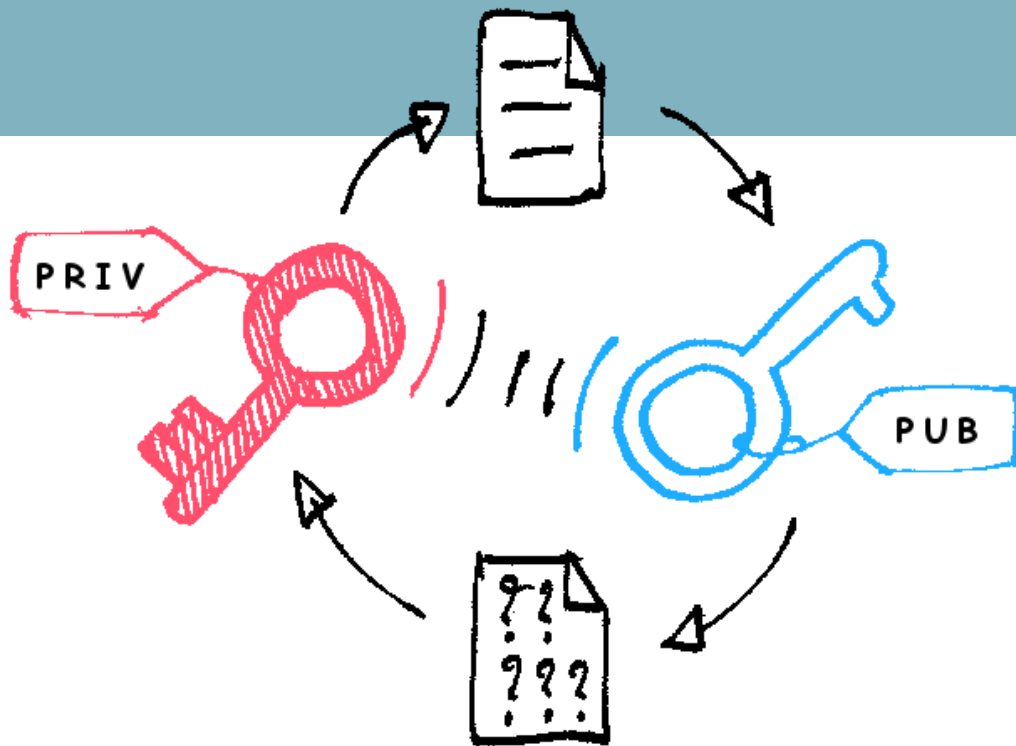




Sesión de Laboratorio 2:

CIFRADO AFÍN



INTRODUCCIÓN A LA CRIPTOGRAFÍA

Profesora || Sandra Díaz Santiago

Grupo || 6CV4

Alumno || Areli Alejandra Guevara Badillo

Septiembre 12, 2025

EJERCICIOS DE PROGRAMACIÓN

EJERCICIO 1

Diseña e implementa una función que reciba como parámetro un entero $n \geq 2$ que cree una lista con los elementos de $\mathbb{Z}_n^*$.

```
1 def ZN_est(n):
2     zn_estrella = []
3     for i in range(1, n):
4         if math.gcd(i, n) == 1:
5             zn_estrella.append(i)
6     return zn_estrella
```

Explicación:

La función recibe un número entero n .

Se mueve evaluando todos los números desde 1 hasta $n-1$, en cada uno de ellos se realiza el modulo con n y si este es igual a 1 (son coprimos) se agrega a la lista *zn_estrella*.

Al terminar de iterar se devuelve nuestra lista ya con todos los elementos de $\mathbb{Z}_n^*$.

EJERCICIO 2

Diseña e implementa una función que reciba como parámetros un entero $n \geq 2$ y un elemento $a \in \mathbb{Z}_n^*$, tu función debe probar cada elemento en $\mathbb{Z}_n^*$ para encontrar y devolver el elemento $b \in \mathbb{Z}_n^*$, tal que $ab \bmod n = 1$.

```
1 def invA(n, a, zn_estrella):
2     for i in zn_estrella:
3         if (i * a) % n == 1:
4             return i
5     return None
```

Explicación:

La función recibe el tamaño de nuestro alfabeto n , nuestra a que pertenece a $\mathbb{Z}_n^*$ y nuestro conjunto $\mathbb{Z}_n^*$.

Esta itera sobre los elementos de *zn_estrella* buscando aquel cuyo módulo de su multiplicación con a modulo n sea igual 1.

Al encontrarlo lo devuelve, en caso contrario devuelve None.

EJERCICIO 3

Diseña e implementa una función que reciba como parámetro el tamaño de un posible alfabeto $n \geq 2$. Tu función debe generar aleatoriamente una clave válida $K=(a, b)$ para el cifrado afín. Usa las funciones que diseñaste para los puntos 1 y 2.

```
1 def clave (n):
2     zn_estrella = ZN_est(n)
3     a = random.choice(zn_estrella)
4     b = random.randint(1, n-1)
5     return a, b
```

Explicación:

La función recibe el tamaño del alfabeto n .

Construye su Z_n^* llamando a la función `ZN_est` y mandándole el tamaño del alfabeto.

Elije un elemento aleatorio de nuestro conjunto `zn_estrella` el cual asigna como a en nuestra clave. Después elige un elemento aleatorio de nuestro conjunto Z_n (de 1 a $n-1$), el cual asigna como la b de nuestra clave.

Finalmente retorna los elementos de la clave (a, b) .

EJERCICIO 6 (A)

Dado el tamaño de un alfabeto n , enumera todas las claves válidas para el cifrado afín, y el correspondiente $a^{-1} \bmod n$. Almacénalas en un archivo de texto.

```
1 def clavesAfines(n):
2     zn_estrella = ZN_est(n)
3
4     with open('claves_afines.txt', 'w') as f:
5         f.write(f"Claves válidas para un alfabeto de tamaño n = {n}\n")
6         f.write("-" * 30 + "\n")
7         f.write("{:<10} {:<10} {:<10}\n".format("a", "b", "a^-1"))
8         f.write("-" * 30 + "\n")
9
10        for a in zn_estrella:
11            a_inverso = invA(n, a, zn_estrella)
12
13            for b in range(n):
14                f.write("{:<10} {:<10} {:<10}\n".format(a, b, a_inverso))
15
16        print(f"Todas las claves válidas se han guardado en el archivo
17        'claves_afines.txt'.")
```

Explicación:

La función recibe nuestro tamaño del alfabeto n .

Construye su Z_n^* llamando a la función `ZN_est` y mandándole el tamaño del alfabeto.

Sesión de Laboratorio 2: Cifrado Afín

Abre un archivo llamado `'claves_afines.txt'` y en él escribe una cabecera y el encabezado de la tabla.

Después itera sobre el conjunto de `zn_estrella` para definir todas las claves a y dentro de este itera sobre el conjunto Z_n para hacer las duplas para cada clave posible. Al mismo tiempo saca el inverso de a llamando a la función `invA` para agregarlas a la tabla.

Conforme va armando las filas las va escribiendo en el archivo.

Una vez termina de iterar en ambas listas guarda el archivo.

Ejecución de los puntos anteriores:

```
--- Menu ---
1. Ejecutar primera parte (Zn*, inverso, clave aleatoria, claves afines)
2. Cifrar texto
3. Descifrar texto
4. Salir
Seleccione una opción: 1
Ingrese un numero: 27
Zn* es:
[1, 2, 4, 5, 7, 8, 10, 11, 13, 14, 16, 17, 19, 20, 22, 23, 25, 26]

Ingrese el valor de a: 16
El inverso de a es: 22

La clave escogida es: (26, 6)
Todas las claves válidas se han guardado en el archivo 'claves_afines.txt'.
```

EJERCICIO 5

Usa el alfabeto de los caracteres imprimibles en el ASCII i.e. $A = \{\text{espacio}, '!', \dots, \sim\}$ con códigos del 32 al 126 y haz lo siguiente:

- a) Diseña e implementa una función para cifrar un texto usando el cifrado afín. Tu función debe recibir una cadena con caracteres en el conjunto A , una clave válida K para el cifrado afín, y devolver el texto cifrado. Tu función debe usar aritmética modular con enteros módulo n , donde n es el tamaño del alfabeto.

```
1 def affine_cipher(A, key1, key2):
2     if isinstance(A, str): A = list(A)
3
4     res = []
5     for char in A:
6         if 32 <= ord(char) <= 126:
7             temp = ord(char) - 32
8             temp2 = (temp * key1 + key2) % 95
9             res.append(chr(temp2 + 32))
10        else:
11            res.append(char)
12
13    return ''.join(res)
```

Sesión de Laboratorio 2: Cifrado Afín

Explicación:

La función recibe nuestro texto plano A y la parte a y b de la clave.

Itera por todos los caracteres de A, verifica si forma parte de los caracteres imprimibles del ASCII (32 - 126), en ese caso le asigna su valor numérico a cada carácter, lo cifra aplicando $(temp * key1 + key2) \% 95$ y lo guarda en res que es nuestro arreglo donde vamos a guardar nuestro texto ifrado.

En caso de que no pertenezca al conjunto del alfabeto no lo cifra y lo agrega tal y como esta.

Al terminar de iterar por toda la palabra devuelve el texto cifrado.

b) Diseña e implementa una función para descifrar un texto cifrado con el cifrado afín.

Tu función debe recibir un texto cifrado (con caracteres en el conjunto A), una clave válida K para el cifrado afín, y recuperar el texto plano original. Usa la función del punto 2 para encontrar $a^{-1} \bmod n$. Tu función debe usar aritmética modular con enteros módulo n, donde n es el tamaño del alfabeto.

```
1 def affine_discipher(A, key1, key2):
2     alfabeto = np.array([chr(i) for i in range(32, 127)])
3
4     if isinstance(A, str):
5         A = list(A)
6
7     res = []
8     for char in A:
9         if char in alfabeto:
10            temp = (ord(char) - 32)
11            inv = invA(95, key1, ZN_est(95))
12            temp2 = ((temp - key2) * inv) % 95
13            res.append(chr(temp2 + 32))
14        else:
15            res.append(char)
16
17    return ''.join(res)
```

Explicación:

La función recibe nuestro texto plano A y la parte a y b de la clave.

Itera por todos los caracteres de A, verifica si forma parte de los caracteres imprimibles del ASCII (32 - 126), en ese caso recupera su valor numérico, consigue la inversa de a mandando a llamar `invA(95, key1, ZN_est(95))`, lo descifra aplicando $((temp - key2) * inv) \% 95$ y lo guarda en res que es nuestro arreglo donde vamos a guardar nuestro texto plano.

En caso de que no pertenezca al conjunto del alfabeto no lo descifra y lo agrega tal y como esta.

Al terminar de iterar por toda la palabra devuelve el texto plano

Sesión de Laboratorio 2: Cifrado Afín

Ejecución:

Cifrado:

```
--- Menú ---
1. Ejecutar primera parte (Zn*, inverso, clave aleatoria, claves afines)
2. Cifrar texto
3. Descifrar texto
4. Salir
Seleccione una opción: 2
Ingrese el texto a cifrar: Hola! ¿Listo para jugar? @$%&/# Sí, claro ;) 1234*
Zn* es:
[1, 2, 3, 4, 6, 7, 8, 9, 11, 12, 13, 14, 16, 17, 18, 21, 22, 23, 24, 26, 27, 28, 29, 31, 32, 33, 34, 36, 37, 39, 41, 42, 43, 44, 46, 47, 48,
9, 81, 82, 83, 84, 86, 87, 88, 89, 91, 92, 93, 94]
Ingrese el valor de a (debe estar en Zn*): 18
Ingrese un valor de b (0 a 94): 5
El texto cifrado es: \"KC7%¿Etj|\"%4CXC%'/PCXx%+m 2u[%dí?%gKCX\"%0h%:L^pz
```

Descifrado:

```
--- Menú ---
1. Ejecutar primera parte (Zn*, inverso, clave aleatoria, claves afines)
2. Cifrar texto
3. Descifrar texto
4. Salir
Seleccione una opción: 3
Ingrese el texto a descifrar: \"KC7%¿Etj|\"%4CXC%'/PCXx%+m 2u[%dí?%gKCX\"%0h%:L^pz
Ingrese el valor de a usado en el cifrado (debe estar en Zn* de 95): 18
Ingrese el valor de b usado en el cifrado: 5
El texto descifrado es: Hola! ¿Listo para jugar? @$%&/# Sí, claro ;) 1234*
```